

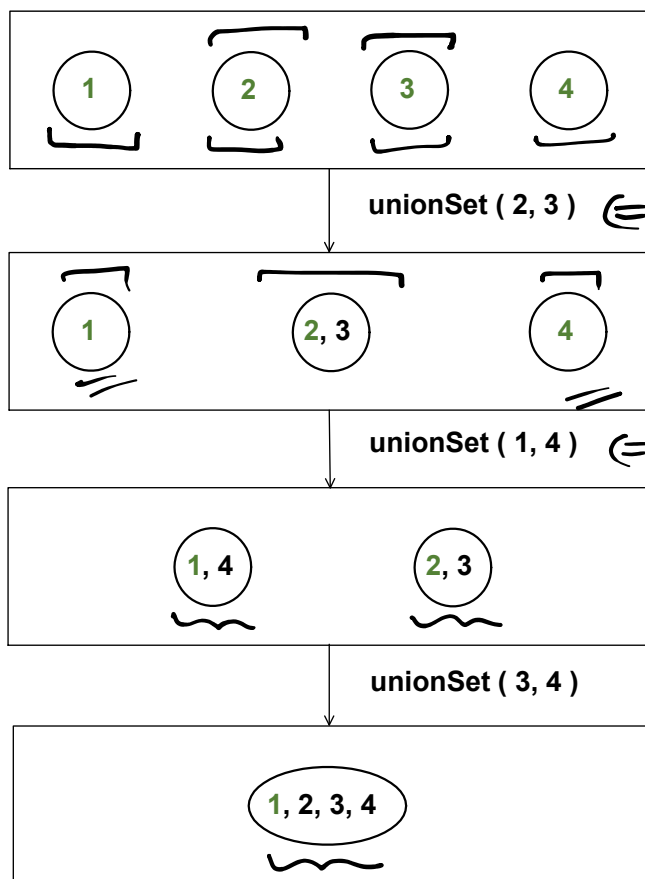
[HW] <https://codeforces.com/edu/course/2/lesson/7/1/practice>
 [HW] <https://cses.fi/problemset/task/1676>

⇒ Disjoint Set

A **disjoint set** is a data structure which maintains a collections of **dynamic** sets $\{S_0, S_1, S_2, \dots\}$ that are pairwise disjoint i.e. for any two sets S_i and S_j in the given collection where $i \neq j$, $S_i \cap S_j = \emptyset$

The disjoint set data structure supports three main operations, these include

- • **createSet (x)**
to create a new set with a single member x (leader/representative of set)
- • **unionSet (x, y)**
to create a set **S** as a union of sets, S_x & S_y that contains x & y respectively.
- • **findSet (x)**
returns the leader or representative of the set that contains the element x.



createSet (1)
 createSet (2)
 createSet (3)
 createSet (4)

findSet (3) → 2

findSet (4) → 1

findSet (1)
 findSet (2)
 findSet (3)
 findSet (4)

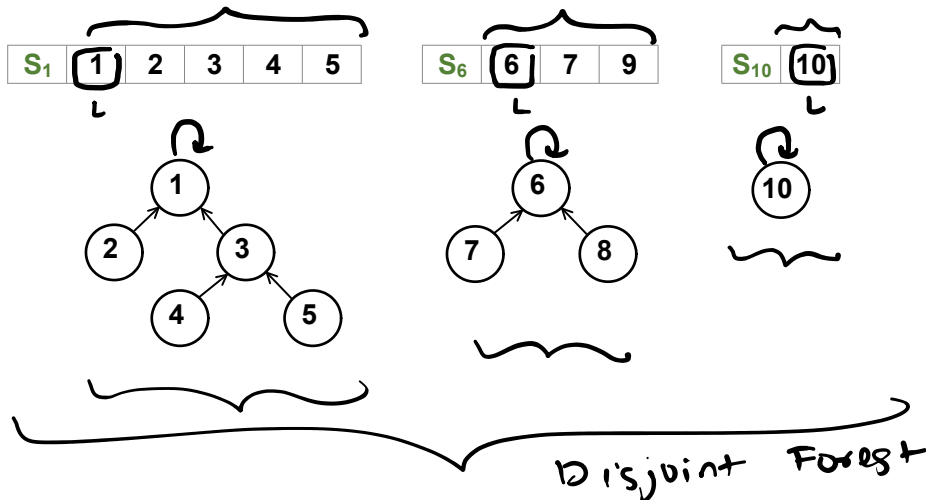
Disjoint Set Representation

There are two main strategies used to represent the disjoint set data structure

- Linked List Representation
- Disjoint Forest Representation (faster)

⇒

In the **Disjoint Forest** representation, we represent each set in the given set collection using a **Rooted Tree** data structure i.e. a tree in which each node points to its parent and each tree node corresponds to a member of the set. Moreover, the **root** node of the tree corresponding to a given set **points to itself** and is the **leader** or the representative of the set.

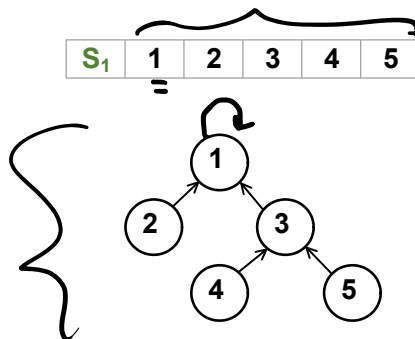


Operations with Forest Representation

- ⇒
- createSet (x)



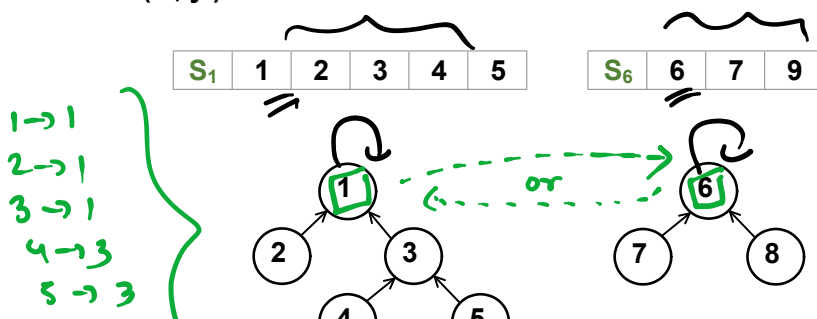
- findSet (x)



time $\propto$ hgt of tree

findSet(5)

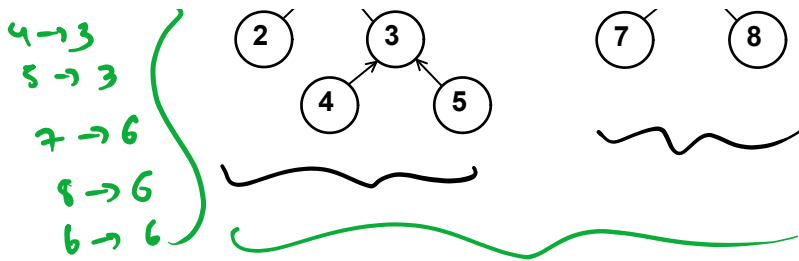
unionSet (x, y)



unionSet(5, 8)

findSet(5) $\rightarrow$ 1

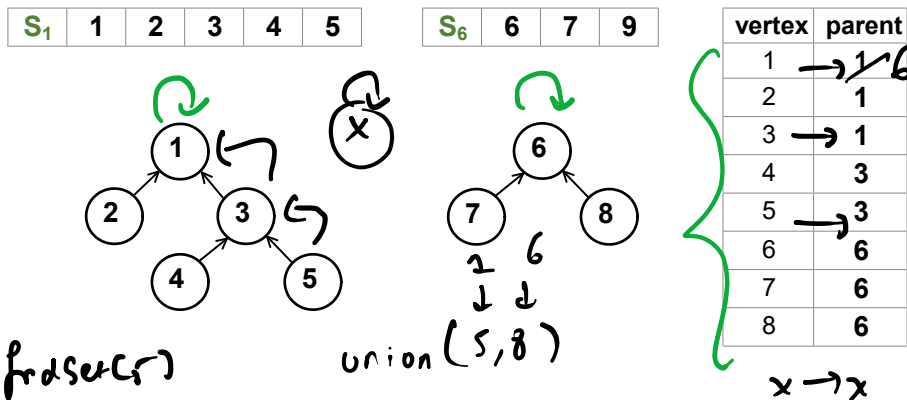
findSet(8) $\rightarrow$ 6



$\text{findset}(5) \rightarrow 1$
 $\text{findset}(8) \rightarrow 6$

Disjoint Set Implementation with Forest Representation

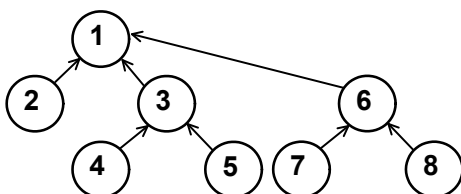
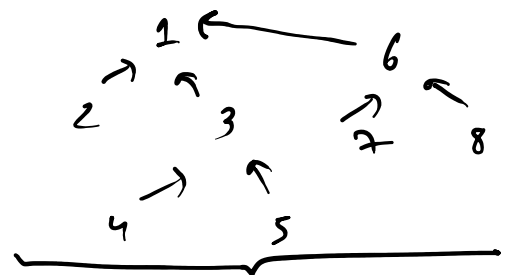
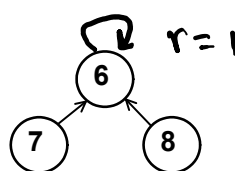
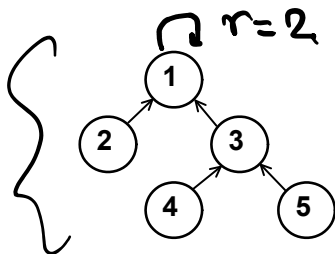
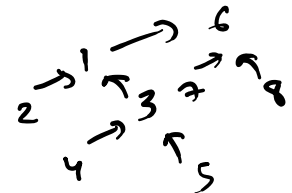
To implement the disjoint set data structure with the forest representation, we will use a **hash table** to store a mapping b/w nodes and their parents.



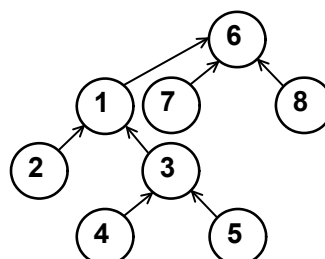
Improving the efficiency of Disjoint Set operations

To reduce the depth of a tree we deploy two heuristics

- **Union by rank (or size)** *(height) → see recording*
- The idea behind union by rank is, during the **union** operation, make the root of the tree with **lower** rank point to root of the tree with **higher** rank.

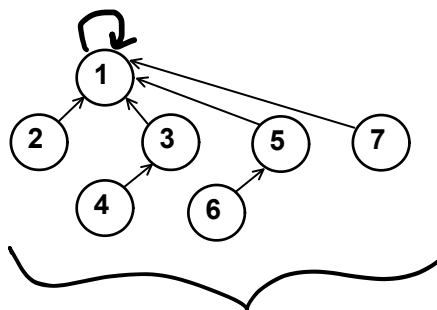
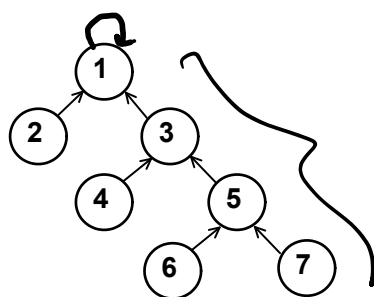


or



- **Path Compression**

The idea behind path compression is, during the **find** operation, make every node visited along the path to the root node, **point to the root**.



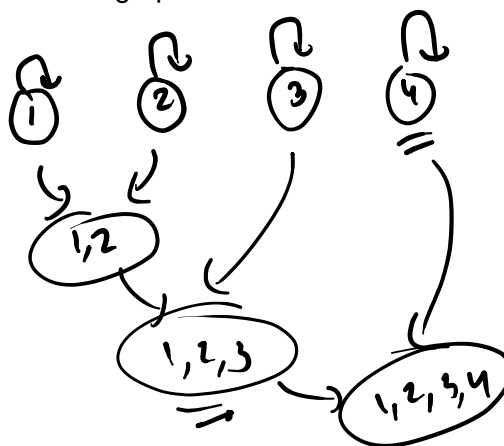
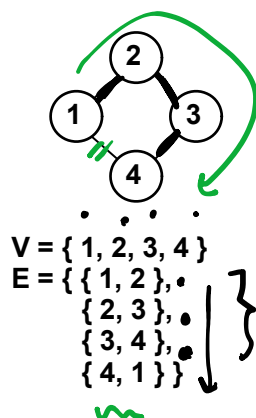
$\text{findSet}(7) \rightarrow 1$

When we **combine** union by rank and path compression, on average the time complexity of the find operation and the union operation is **constant**.

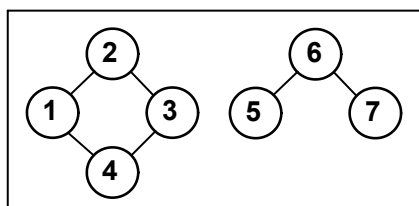
$\alpha(n) \sim < 5$ for v.v.v. large n
 $\downarrow$
 inv. ackerman fn

Applications of Disjoint Set

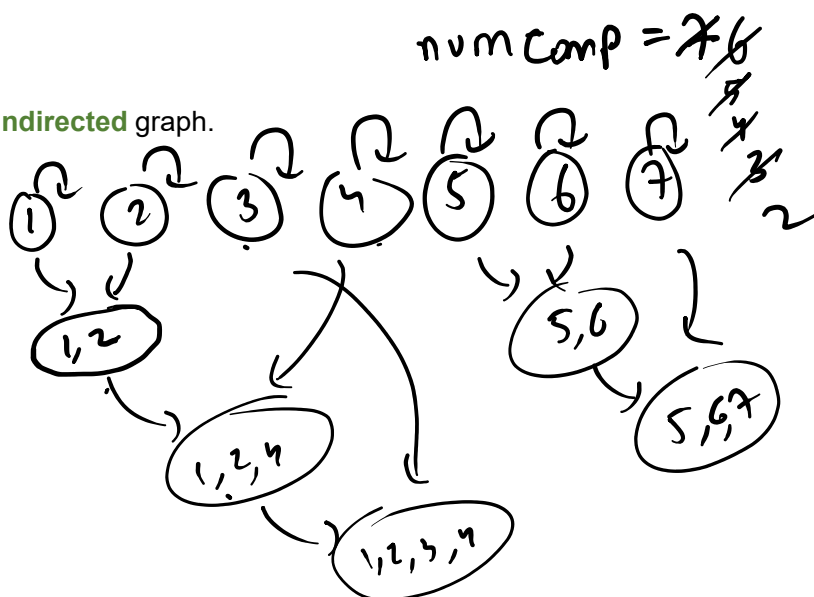
- It can be used to **detect a cycle** in an **undirected** graph.



- It can be used to **count components** in an **undirected** graph.



$V = \{1, 2, 3, 4, 5, 6, 7\}$
 $E = \{\{1, 2\}, \{1, 4\}, \{2, 3\}, \{3, 4\}, \{5, 6\}, \{6, 7\}\}$

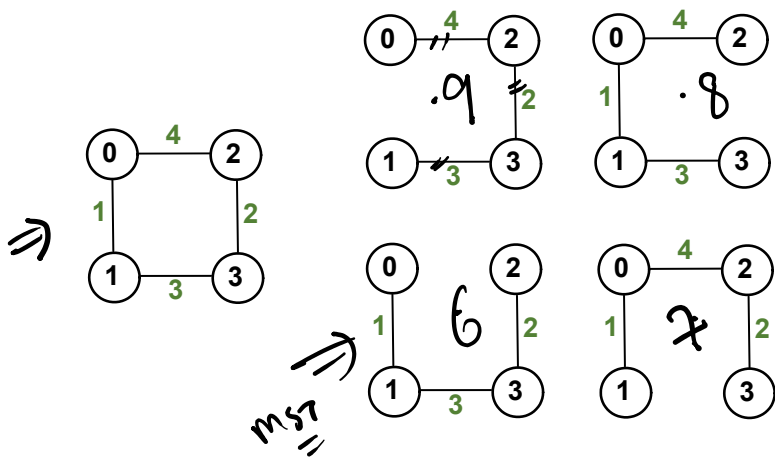
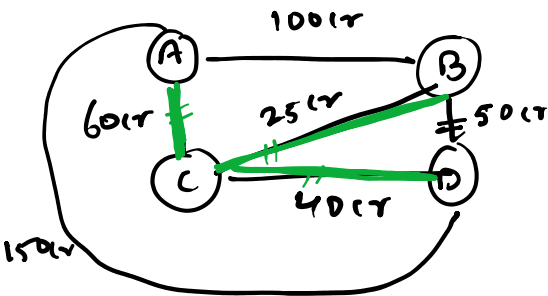


⇒ In a tree with n nodes there will be $n-1$ edges

[HW] <https://codeforces.com/problemset/problem/1559/D1>

Minimum Spanning Tree (MST)

Given a **connected, undirected, weighted** graph, design an algorithm to its find the **minimum spanning tree**.



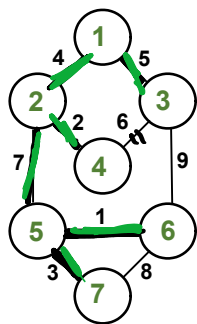
Kruskal ✓ (dsu)
Prim (minHeap)
[R]

Kruskal's Algorithm

The idea behind the **Kruskal's** algorithm to find the **MST** for the given **connected, undirected, weighted** graph that contains **V** vertices is to choose **V-1** edges in the **increasing** order of their weights such that at no point of time inclusion of an edge in the sub-graph leads to a **cycle**.

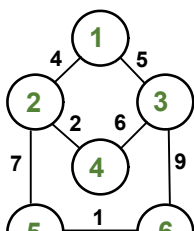
$n = 7$

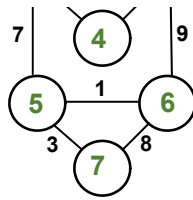
$n - 1$



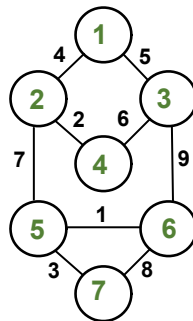
- 5-6 : 1 ✓
- 2-4 : 2 ✓
- 5-7 : 3 ✓
- 1-2 : 4 ✓
- 1-3 : 5 ✓
- 2-6 : 7 ✓

Implementation of Kruskal's Algorithm

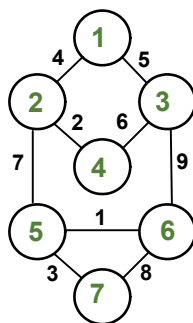




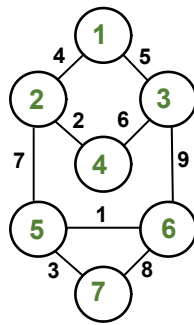
[R][HW] Prim's Algorithm



Implementation of Prims Algorithm



Optimized Implementation of Prims Algorithm



Minimize Malware Spread

You are given a network of `n` nodes represented as an `n x n` adjacency matrix `graph`, where the `ith` node is directly connected to the `jth` node if `graph[i][j] == 1`.

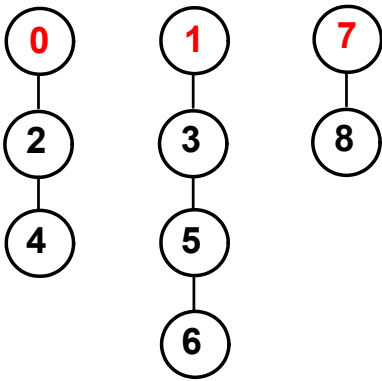
Some nodes `initial` are initially infected by malware. Whenever two nodes are directly connected, and at least one of those two nodes is infected by malware, both nodes will be infected by malware. This spread of malware will continue until no more nodes can be infected in this manner.

Suppose `M(initial)` is the final number of nodes infected with malware in the entire network after the spread of malware stops. We will remove **exactly one node** from `initial`.

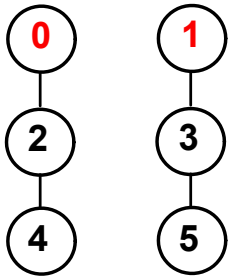
Return the node that, if removed, would minimize `M(initial)`. If multiple nodes could be removed to minimize `M(initial)`, return such a node with **the smallest index**.

Note that if a node was removed from the `initial` list of infected nodes, it might still be infected later due to the malware spread.

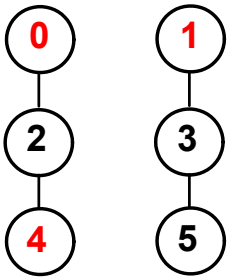
	0	1	2	3	4	5	6	7	8
0	1	0	1	0	0	0	0	0	0
1	0	1	0	1	0	0	0	0	0
2	1	0	1	0	1	0	0	0	0
3	0	1	0	1	0	1	0	0	0
4	0	0	1	0	1	0	0	0	0
5	0	0	0	1	0	1	1	0	0
6	0	0	0	0	0	1	1	0	0
7	0	0	0	0	0	0	0	1	1
8	0	0	0	0	0	0	0	1	1



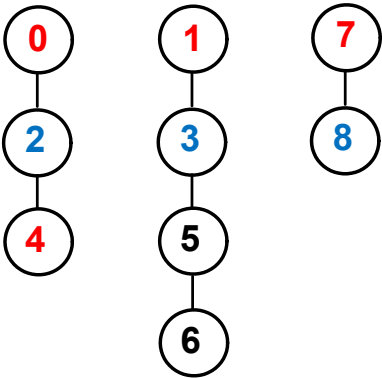
	0	1	2	3	4	5
0	1	0	1	0	0	0
1	0	1	0	1	0	0
2	1	0	1	0	1	0
3	0	1	0	1	0	1
4	0	0	1	0	1	0
5	0	0	0	1	0	1



	0	1	2	3	4	5
0	1	0	1	0	0	0
1	0	1	0	1	0	0
2	1	0	1	0	1	0
3	0	1	0	1	0	1
4	0	0	1	0	1	0
5	0	0	0	1	0	1



	0	1	2	3	4	5	6	7	8
0	1	0	1	0	0	0	0	0	0
1	0	1	0	1	0	0	0	0	0
2	1	0	1	0	1	0	0	0	0
3	0	1	0	1	0	1	0	0	0
4	0	0	1	0	1	0	0	0	0
5	0	0	0	1	0	1	1	0	0
6	0	0	0	0	0	1	1	0	0
7	0	0	0	0	0	0	0	1	1
8	0	0	0	0	0	0	0	1	1



0	1	2	3	4	5	6	7	8
2	3	2	3	2	3	3	8	8

parentMap

0	1	2	3	4	5	6	7	8
		3	4					2

rankMap